Algoritmos Genéticos

TP2 — Sistemas de Inteligencia Artificial

Aproximación de Imágenes con Triángulos mediante Evolución

ITBA · 2026

Índice

1 Introducción: Algoritmos Genéticos

2 El Problema: Aproximación con Triángulos

3 Representación del Individuo

4 Función de Aptitud

5 Selección

6 Cruza (Crossover)

7 Mutación

8 Supervivencia y Diversidad

9 Resultados y Análisis

10 Conclusiones

Sección 1

Algoritmos Genéticos

Inspiración biológica aplicada a optimización

¿Qué son los Algoritmos Genéticos?

Metaheurística de búsqueda inspirada en la evolución natural de Darwin.

Población	Conjunto de soluciones candidatas (individuos).
Gen / Cromosoma	Representación codificada de una solución.
Fitness	Medida de qué tan buena es una solución.
Selección	Los más aptos tienen mayor probabilidad de reproducirse.
Cruza	Combinación de genes de dos padres para generar descendencia.
Mutación	Perturbación aleatoria que introduce variedad genética.
Supervivencia	Mecanismo que determina quiénes pasan a la siguiente generación.

El Ciclo Evolutivo

Cada generación repite el mismo ciclo hasta alcanzar el criterio de parada:

1. Inicialización	Generar población inicial aleatoria (o guiada).
2. Evaluación	Calcular el fitness de cada individuo.
3. Selección	Elegir padres según su aptitud relativa.
4. Cruza	Combinar pares de padres $\longrightarrow$ offspring.
5. Mutación	Perturbar genes del offspring aleatoriamente.
6. Supervivencia	Construir la nueva generación (elitismo + offspring).
7. Terminación	Verificar condición de parada. Si no $\longrightarrow$ volver a 2.

Sección 2

El Problema

Aproximación de imágenes mediante triángulos semitransparentes

Ejercicio 1 — ASCII Art (conceptual)

Representar una imagen cuadrada como un mapa NxN de caracteres ASCII.

Individuo

→ Grilla de R×C caracteres

→ Cada celda = índice en charset

→ charset: '@%#*+=-:. ' (oscuro→claro)

Fitness

→ Renderizar grilla con fuente monoespaciada

→ Comparar con imagen target en escala de grises

→ fitness = MSE(render, target)

Mutación

→ 70% probabilidad: paso ±1 en ranking de oscuridad

→ 30% probabilidad: carácter completamente aleatorio

→ Mutation rate baja (~0.02) para estabilidad

Implementado — warm start con greedy brightness mapping + GA sobre él

Ejercicio 2 — Aproximación con Triángulos

Dado N triángulos semitransparentes (RGBA) sobre canvas blanco, ¿cómo replicar la imagen original?

Parámetros de entrada

→ Imagen objetivo (cualquier resolución)

→ N = cantidad de triángulos

→ Hiperparámetros del AG (población, generaciones, etc.)

Salida

→ Imagen generada (PNG)

→ Lista de triángulos con posición y color

→ Métricas: fitness, error, evolución por generación

Desafíos

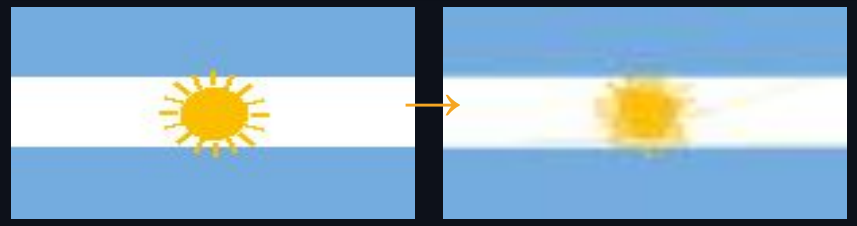
→ Espacio de búsqueda continuo y de alta dimensión

→ Muchos mínimos locales (orden de pintado importa)

→ Fitness costoso: renderizar imagen en cada evaluación

→ Balance exploración / explotación crítico

→ Convergencia prematura si no hay diversidad



Sección 3

Representación del Individuo

Cromosoma, genes y espacio de búsqueda

Genotipo: Representación del Individuo

Cada individuo es una lista de N triángulos codificada como una matriz float32.

Cromosoma: array float32 de forma (N, 10)

Locus 1

x_1	y_1	x_2	y_2	x_3	y_3	R	G	B	α
Vértices						Color RGB + Alpha			

...

Locus N

Dimensión total	N × 10 genes reales (ej: 100 triángulos ⟶ 1000 genes)
Rango de valores	Todos los genes normalizados en [0, 1]
Renderizado	Desnormalización en tiempo de render: coords × tamaño canvas, color × 255
Alternativa	Óvalo: (cx, cy, rx, ry, R, G, B, α) ⟶ N × 8 genes
Inicialización	50% random · 50% color-sampled desde la imagen objetivo (warm start)

Sección 4

Función de Aptitud

Cómo medimos la calidad de cada individuo

Función de Aptitud (Fitness)

Buscamos minimizar el error entre la imagen generada y la imagen objetivo.

fitness(individuo) = MSE(render(cromosoma), imagen_objetivo)

MSE = mean((píxel_generado − píxel_real)²) → menor es mejor → óptimo teorico = 0

MSE estándar

→ Todos los píxeles contribuyen igual

→ Simple y eficiente computacionalmente

→ Puede ignorar detalles perceptualmente importantes

→ Adecuado para imágenes uniformes (banderas, logos)

MSE con peso de Saliencia (saliency_weight=0.3)

→ Pondera más los píxeles brillantes y saturados

→ saliency = brillo^1.5 × saturación

→ Focaliza el esfuerzo evolutivo en regiones destacadas

→ Recomendado para fotos complejas (La Noche Estrellada…)

Sección 5

Selección

¿Quiénes se reproducen? La presión selectiva y el balance exploración/explotación

Métodos de Selección — Visión General

Método	Estrategia	Presión Selectiva	Característica
Elite	Top-k directamente	Muy alta	Determinista, sin aleatoriedad
Ruleta	Prob. proporcional al fitness	Alta	Alta varianza; puede dominar un individuo
Universal (SUS)	Punteros equiespaciados en ruleta	Alta	Baja varianza, más uniforme que ruleta
Boltzmann	Temp. T decae con generación	Variable	Exploración al inicio → explotación al final
Torneo Det.	Mejor de k individuos aleatorios	Configurable	Robusto, independiente de escala de fitness
Torneo Prob.	Mejor de k gana con probabilidad p	Configurable	Introduce ruido, evita convergencia prematura
Ranking	Prob. basada en rango, no en valor	Moderada	Estabiliza cuando fitnesses son similares

★ *Configuración recomendada: Boltzmann para exploración inicial + Torneo Determinístico para refinamiento.*

Selección de Boltzmann — Annealing Evolutivo

Inspirada en el recocido simulado: la temperatura T decrece con el tiempo.

$$P(i) \propto \exp(-\text{fitness}(i) / T(g)) \quad \text{donde} \quad T(g) = T_max - (T_max - T_min) \times g / G_max$$

Fase Exploratoria (T alto)

→ T ≈ 100 → diferencias de fitness se aplanan

→ Todos los individuos tienen probabilidades similares

→ Alta diversidad, exploración amplia del espacio

→ Permite escapar de mínimos locales

Fase de Explotación (T bajo)

→ T ≈ 1.0 → diferencias se amplifican

→ El mejor individuo concentra casi toda la probabilidad

→ Convergencia rápida hacia el mejor encontrado

→ Equivalente a selección por ruleta con alta presión

Implementación: T_max=100, T_min=1.0, decae linealmente sobre G_max generaciones.

Sección 6

Cruza (Crossover)

Combinación de cromosomas para generar descendencia

Métodos de Cruza

La cruza opera a nivel de locus (triángulo). Cada fila del cromosoma se hereda completa de uno de los padres.

Uniforme

(uniform)

→ Cada triángulo heredado de P1 o P2 con prob. 0.5

→ Máxima mezcla genética

→ Mejor para combinar características independientes

→ Puede romper bloques de triángulos complementarios

Un punto

(one_point)

→ Punto de corte aleatorio en [1, N-1]

→ C1 = P1[:k] + P2[k:]

→ Preserva bloques contiguos de triángulos

→ Bueno cuando la posición en el cromosoma importa

Dos puntos

(two_point)

→ Dos puntos de corte k1 < k2

→ C1 = P1[:k1] + P2[k1:k2] + P1[k2:]

→ Intercambia segmento interno

→ Balance entre mezcla y preservación de estructura

Anular

(annular)

→ Como dos puntos pero el segmento puede 'dar la vuelta'

→ Todos los segmentos tienen igual probabilidad

→ Elimina el sesgo de posición de los extremos

→ Más robusto que dos puntos en genomas circulares

★ *Recomendación: Uniforme para imágenes complejas (mayor diversidad). Un punto/anular para convergencia más rápida.*

Sección 7

Mutación

Introducción de variedad genética para evitar estancamiento

Métodos de Mutación

La mutación perturba los genes con ruido gaussiano N(0, σ × escala_por_tipo_de_gen).

Método	Genes afectados	Cuándo usarlo
Gen	1 gen aleatorio por individuo	Fine-tuning, generaciones avanzadas
MultiGen	1 a max_genes genes aleatorios (default: 5)	Exploración moderada
Uniforme	Cada gen con prob. mutation_rate (σ fijo)	Exploración amplia y constante
No Uniforme	Como uniforme pero σ decae con la generación	★ Recomendado: exploración→explotación

Mutación No Uniforme — Detalle

$$\sigma(g) = \max(\sigma_min,\ \sigma_0 \times (1 - g/G_max)^2)$$

→ Al inicio (g≈0): σ ≈ σ_0 → grandes saltos → exploración amplia

→ Al final (g≈G): σ ≈ σ_min → pequeñas perturbaciones → refinamiento local

→ σ_min evita que la mutación se apague completamente (floor de diversidad)

Bonus: mutación de orden de capas (swap de orden de pintado de triángulos → afecta composición alpha).

Sección 8

Supervivencia y Diversidad

¿Quiénes forman la siguiente generación?

Estrategias de Supervivencia

¿Cómo se forma la nueva generación a partir de padres y descendencia?

Supervivencia Exclusiva

(Generacional)

Nueva generación = élite + offspring. Los padres NO compiten (excepto los élite).

Ventajas:

✓ Alta presión selectiva

✓ Convergencia rápida

✓ Evita retrocesos de fitness

Desventajas:

✗ Puede perder diversidad

✗ Más propenso a convergencia prematura

Supervivencia Aditiva

(Steady-State)

Pool = padres $\cup$ offspring, se conservan los mejores N individuos.

Ventajas:

✓ Preserva buenos padres

✓ Mayor diversidad en pool

✓ Elitismo implícito

Desventajas:

✗ Menor presión selectiva

✗ Convergencia más lenta

✗ Computacionalmente algo más costoso

Diversidad y Criterios de Terminación

Mecanismos para evitar convergencia prematura y definir cuándo detener la evolución.

Reinicio por Diversidad

→ Detecta colapso de diversidad: std(fitness) < umbral

→ Reemplaza la fracción peor de la población (30%) con individuos frescos

→ Cooldown de 20 generaciones entre reinicios

→ El élite siempre se preserva → nunca pierde el mejor encontrado

→ Activa cuando la población converge prematuramente

Criterios de Terminación

Generaciones máximas	generations=2000 por defecto
MSE objetivo	stop si best_mse ≤ target_mse
Estancamiento	sin mejora > δ en N generaciones
Convergencia	std(fitness) < threshold

En la práctica: max_generations es el más usado. Los otros sirven para runs automáticos largas.

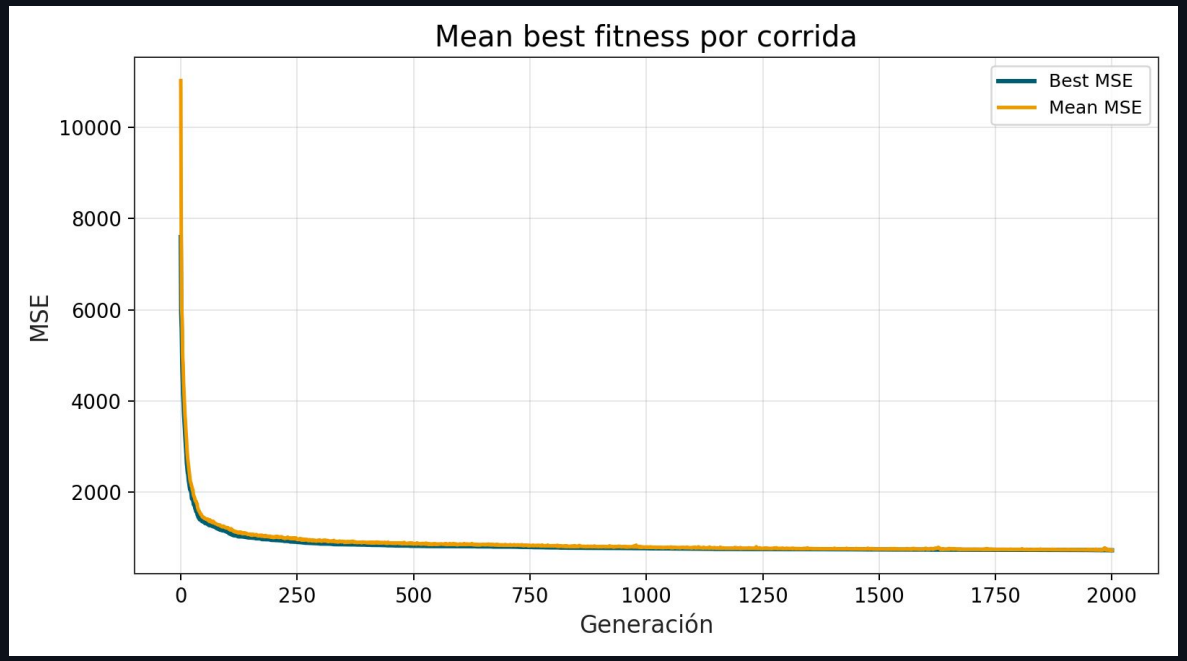
Configuración de defaults: pop=150, gens=2000, elite=5, Boltzmann, non_uniform mutation, diversity restart ON.

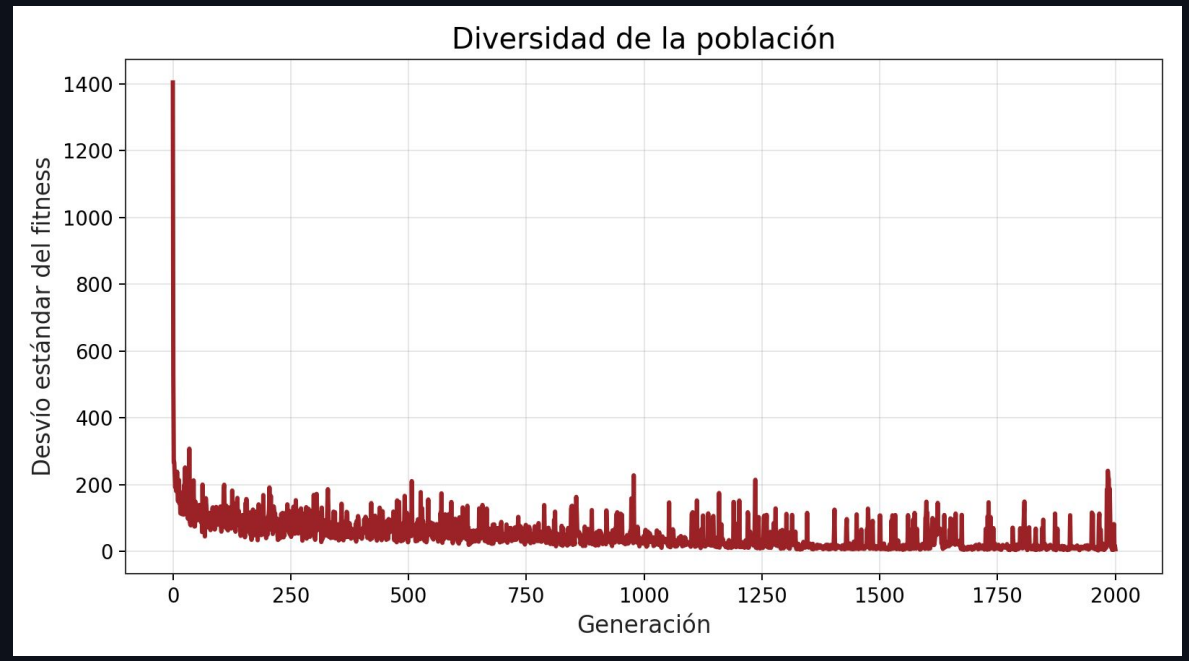
Sección 9

Resultados y Análisis

Evolución visual y comparación de operadores genéticos

Evolución de una corrida





Evolución Visual — Argentina (35k generaciones)

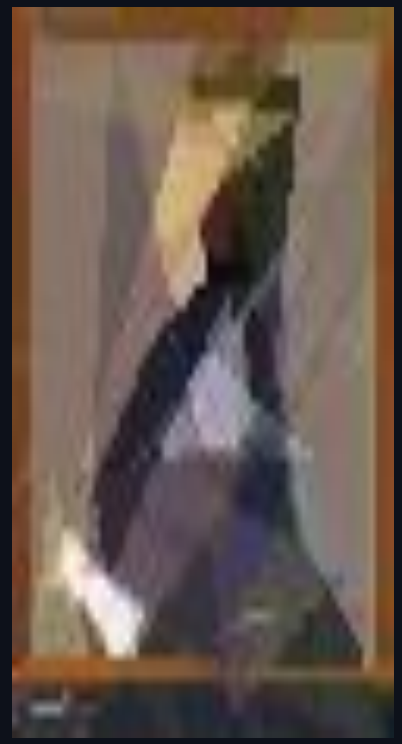
Snapshots de la evolución: de ruido aleatorio a una representación fiel.

Gen 5

Gen 500

Gen 2000

Gen 5000



Gen 10000

Final

Imagen original →

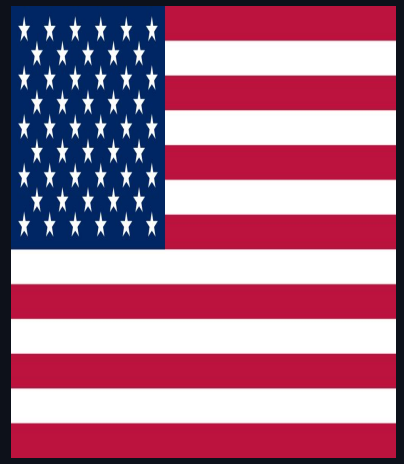
Galería de Resultados

Distintas imágenes procesadas con el motor de AG (100 triángulos, configuración default).

Argentina (35k gens)

Australia (20k gens)

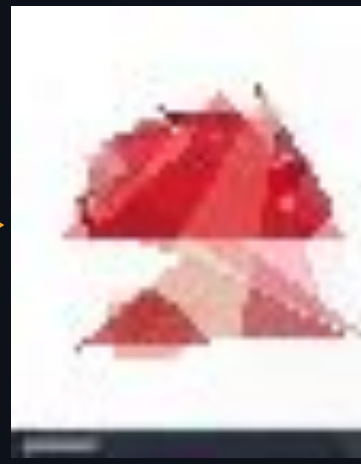
China (20k gens)



USA (20k gens)

Israel (24k gens)

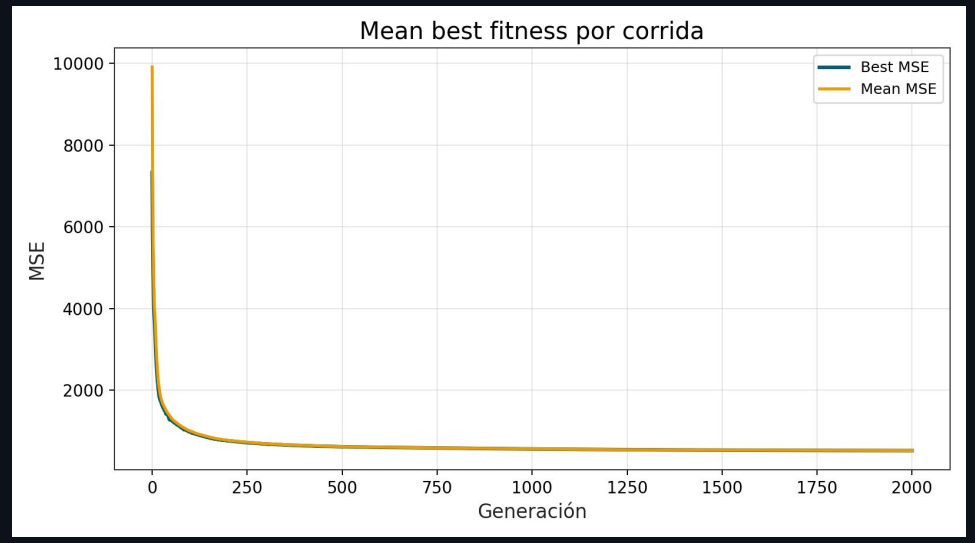
Mushroom (19.5k gens)



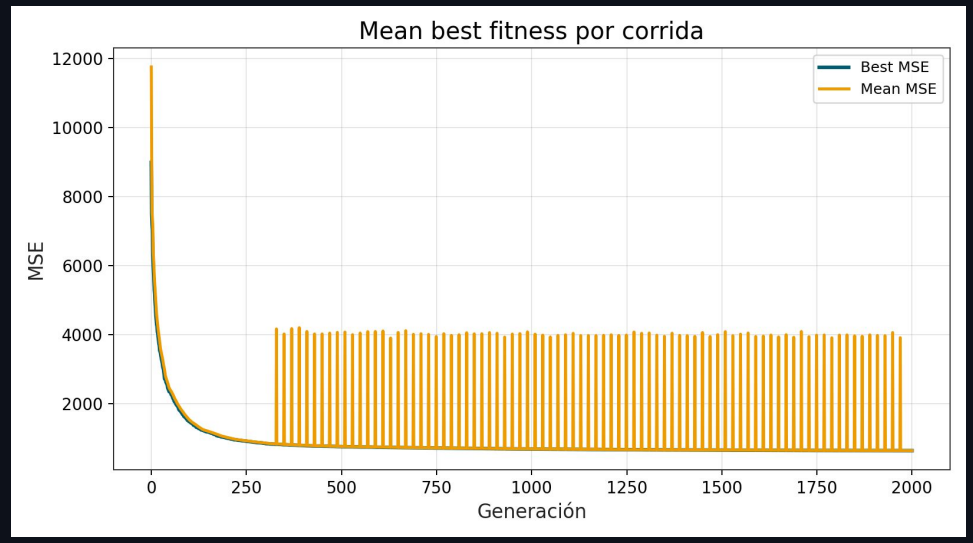
Impacto del diversity restart

Sin diversity restart el algoritmo se estanca; con restart se reintroduce diversidad y continúa la exploración

sin diversity restart

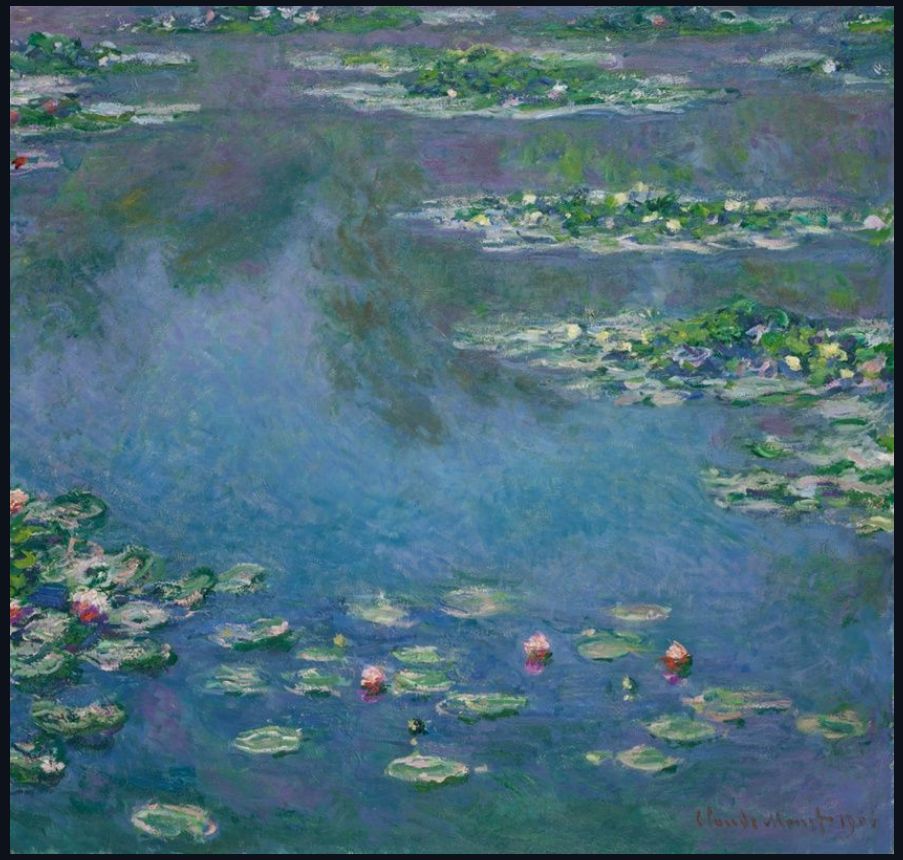


con initial population random (sin sampling) pero diversity restart



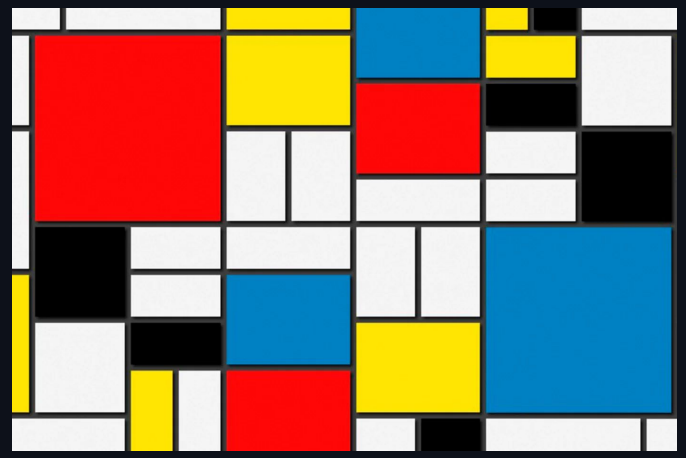
Aproximación de imagen con detalles finos

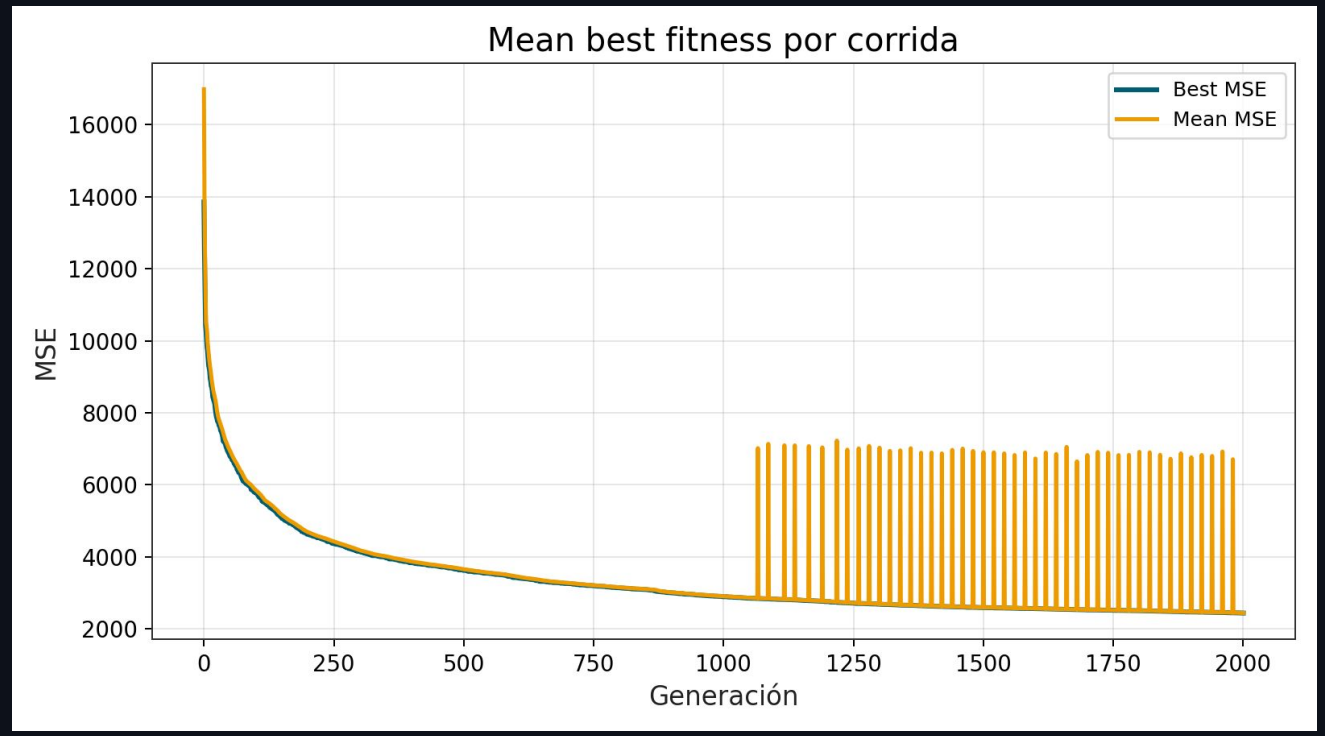
Las formas y colores macro los identifica bien, para detalles minuciosos es más sensible.



Limitación de la representación en imágenes con formas rectangulares

Los triángulos capturan bien el color y la estructura global, pero no representan con precisión bordes rectos y regiones rectangulares.



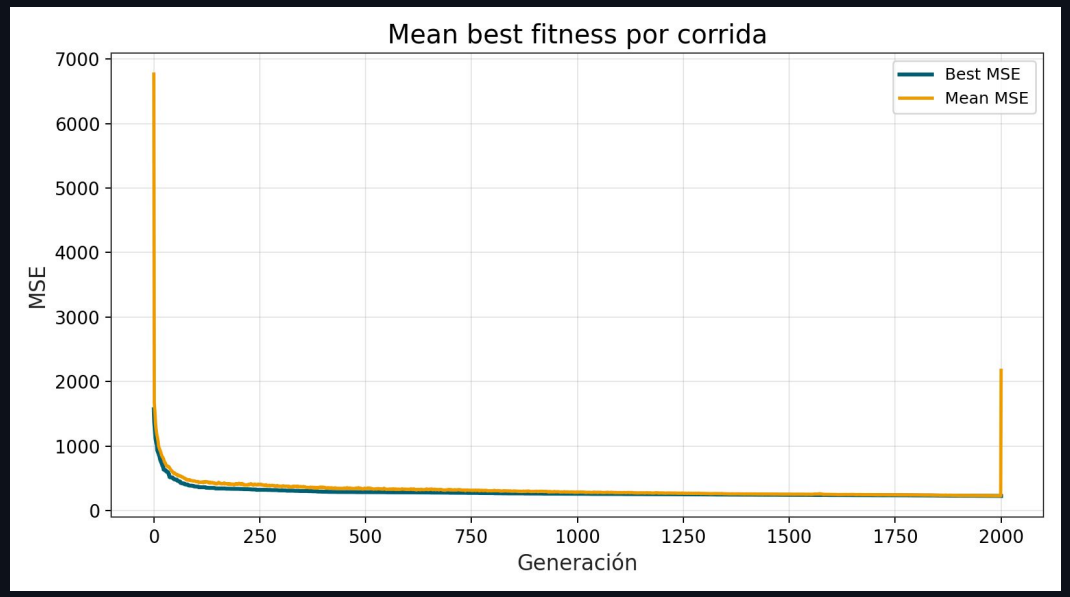


Comparación de representaciones: óvalos vs triángulos

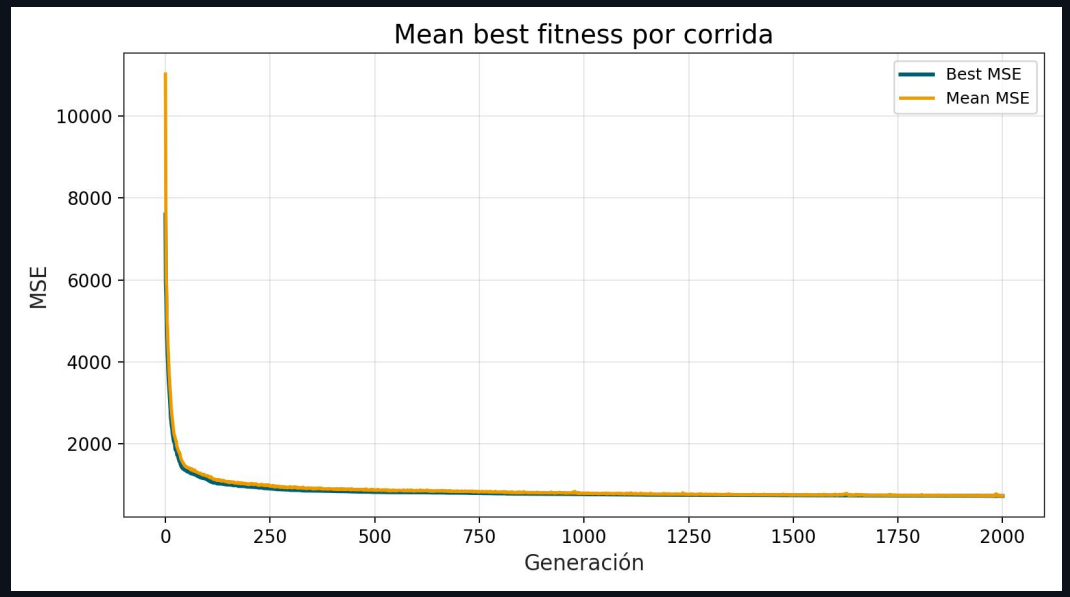
python3 triangles_ga/main.py images/dog.png --n-triangles 80 --img-size 150
--population 120 --generations 1500 --elite 5 --init color_sample --survival
additive --saliency-weight 0.5 --workers 0

Mismo comando de ejecución que anterior, pero con shape oval

Con óvalos se alcanza un fitness similar en menos generaciones, mostrando una convergencia más rápida.



óvalos



triángulos

Análisis Comparativo de Operadores

Tests sistemáticos sobre smiley y bandera argentina — 100 gens, pop=150, 64×64px.

Dimensión	Mejor opción observada	Por qué
Selección	Boltzmann	Equilibra exploración inicial con explotación final de forma automática
Cruza	Uniforme	Mayor mezcla genética, especialmente útil en imágenes complejas y con muchos triángulos
Mutación	No Uniforme	Reduce σ adaptativamente; refinamiento fino sin perder exploración temprana
Supervivencia	Aditiva + élite	Preserva padres buenos, mantiene diversidad, sin regresión de fitness
Diversidad	Restart ON	Evita convergencia prematura en runs largos; esencial con poblaciones pequeñas

Nota: los resultados óptimos dependen de la imagen. Imágenes simples (banderas) convergen rápido con cualquier combinación.

Métricas registradas por generación: best_mse, mean_mse, std_mse → visualización con matplotlib y CSV.

Arquitectura del Sistema

El motor de AG está desacoplado en módulos independientes y reutilizables.

config.py	Dataclass de configuración con validación y valores por defecto
genome.py	Inicialización de cromosomas: random, color-sampled, mixed
render.py	Backends de renderizado: Skia (5-15× más rápido) + PIL (fallback)
fitness.py	MSE + saliency weighting + subsampling de píxeles
operators.py	6 selecciones × 4 cruzas × 4 mutaciones = 96 combinaciones
ga.py	Bucle evolutivo principal: evaluación paralela, élite, restart
io.py	Guardado de snapshots, JSON, métricas CSV, metadatos
main.py	CLI con argparse: todos los hiperparámetros configurables

```
python triangles_ga/main.py imagen.jpg --n-triangles 100 --selection boltzmann --mutation non_uniform
```

Sección 10

Conclusiones

Conclusiones

Representación continua funciona

Codificar triángulos como genes reales en [0,1] permite a los operadores estándar de AG trabajar eficazmente sobre geometría y color.

Boltzmann + No Uniforme = mejor combinación

La annealing natural de ambos operadores (temperatura y sigma) se complementa: exploración amplia al inicio, refinamiento fino al final.

La función fitness es el cuello de botella

Renderizar cada individuo es costoso → paralelización en múltiples núcleos es fundamental. Subsampling de píxeles ofrece speedup con mínima pérdida de calidad.

Diversidad es crítica en runs largos

El reinicio por diversidad evita la convergencia prematura que de otro modo bloquea la evolución en mínimos locales, especialmente con imágenes complejas.

Más triángulos ≠ necesariamente mejor

Más triángulos amplían el espacio de búsqueda y elevan el costo de evaluación. Para imágenes simples, 50-100 triángulos son suficientes.

Imagen inicial importa: warm start acelera

Inicializar con colores muestreados de la imagen objetivo reduce significativamente las generaciones necesarias para alcanzar una buena aproximación.

¡Gracias!

Algoritmos Genéticos — TP2 · Sistemas de Inteligencia Artificial